

Efficiency of Pairing Heaps

Robert E. Tarjan, Xiaoyang Xu
ret@cs.princeton.edu, xiaoyangxu@ucsb.edu

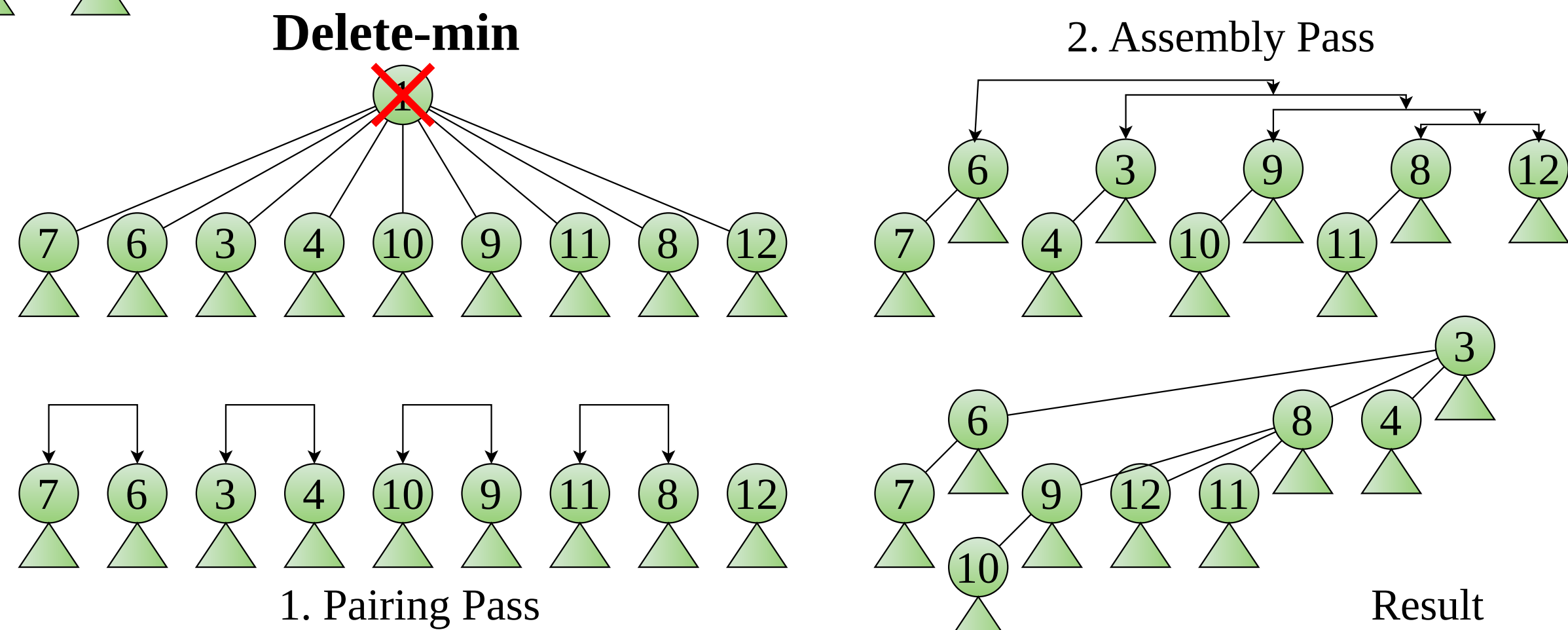
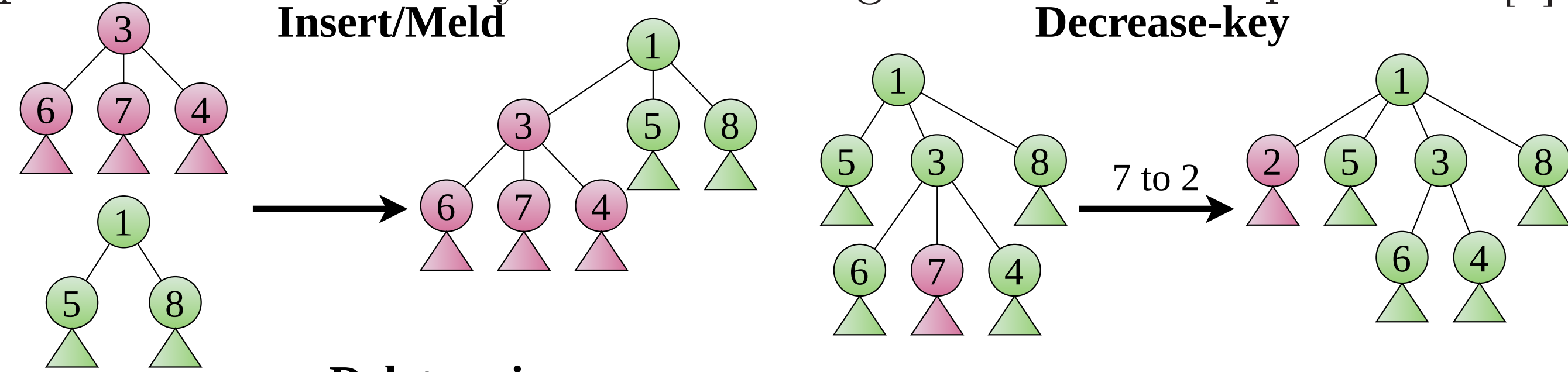


PRINCETON
UNIVERSITY

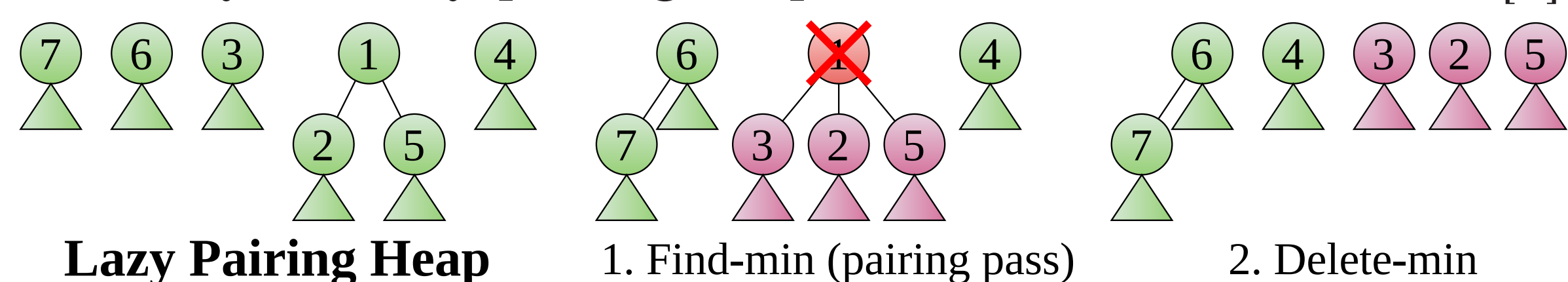
UC SANTA BARBARA
Computer Science

Problem

We analyze the time complexity of pairing heaps, widely used priority queues whose efficiency still lacks a tight theoretical explanation [1]:



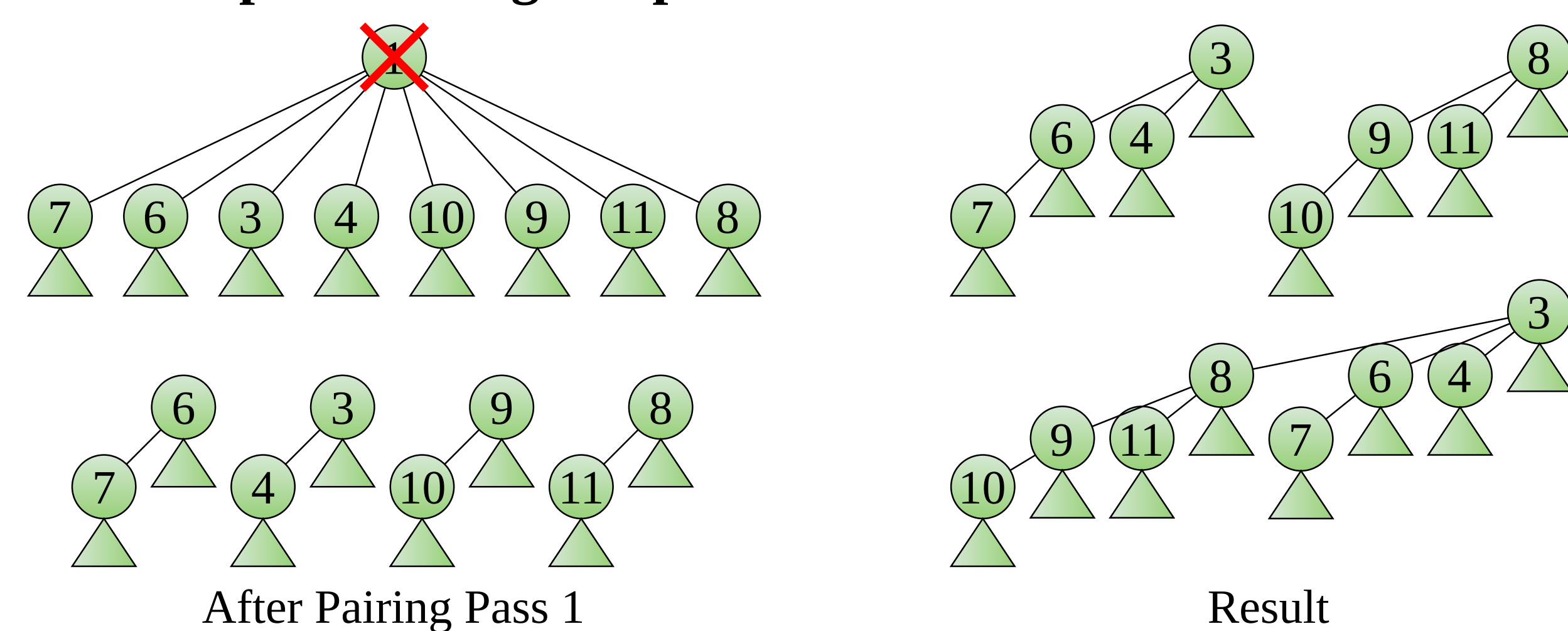
We also study the lazy pairing heaps which maintain a forest [1]:



Background

The efficiency bounds on multipass pairing heap is near tight [1, 2]

Multipass Pairing Heap

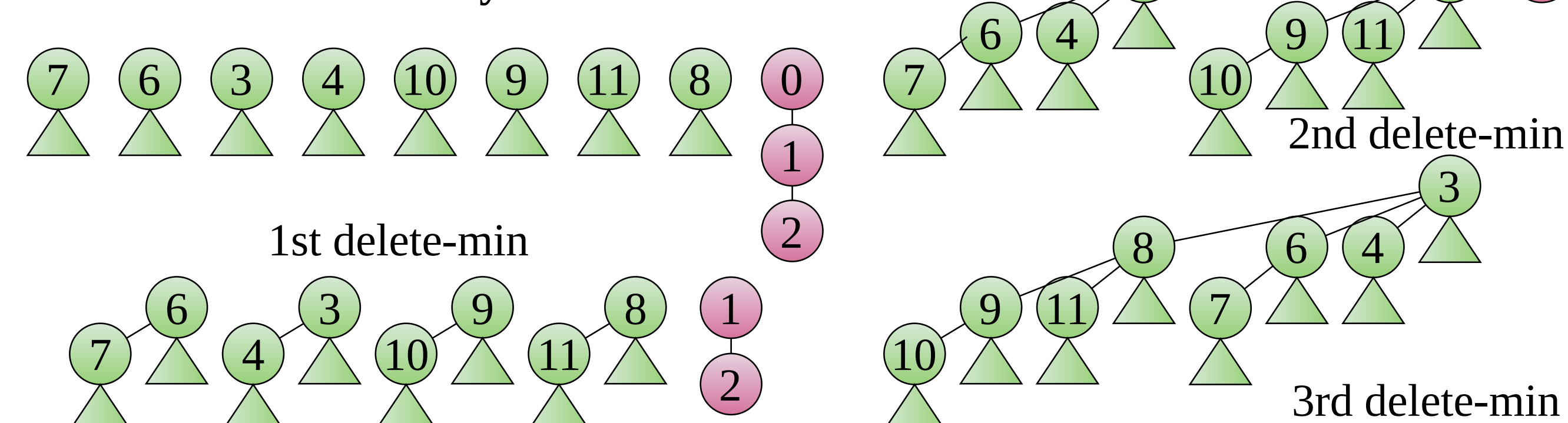


Our analysis on pairing heaps is inspired by the multipass analysis [2].

Intuition

(Lazy) Pairing heaps resemble multipass pairing heaps in this case:

An Ideal Case of Lazy Delete-min



References

- [1] Michael L. Fredman, Robert Sedgwick, Daniel D. Sleator and Robert E. Tarjan *The pairing heap: A new form of self-adjusting heap*, Algorithmica (1986)
- [2] Corwin Sinnamon and Robert E Tarjan: *Efficiency of Self-Adjusting Heaps*, ACM Transactions on Algorithms (2025)
- [3] Michael L. Fredman: *On the Efficiency of Pairing Heaps and Related Data Structures*, Journal of the ACM (1999)

Acknowledgements

This research is partially supported by a gift from Microsoft.

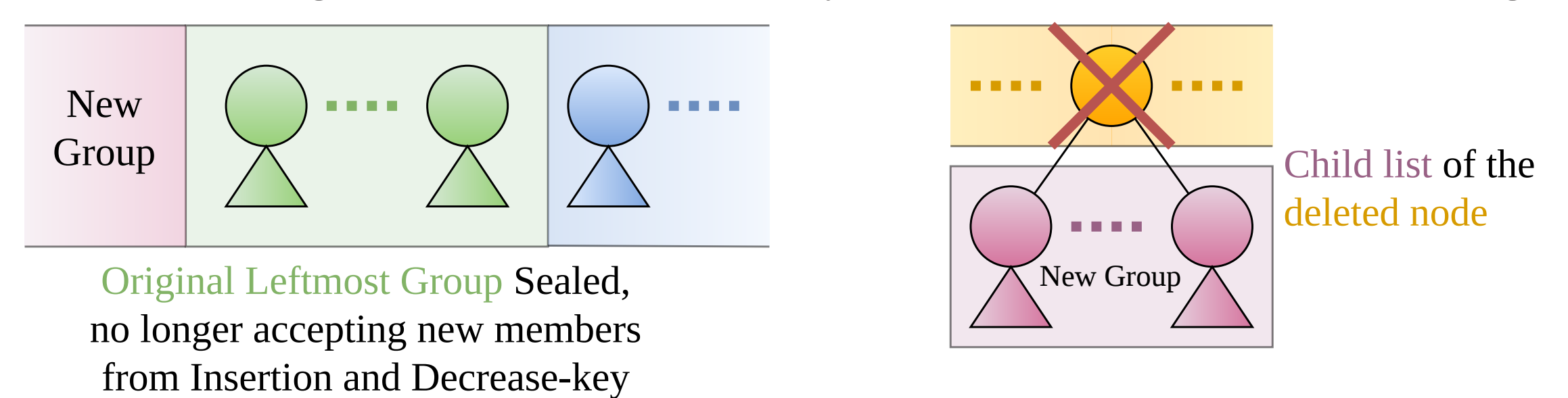
Main Results

Work	INSERT/MELD	DECREASE-KEY	DELETE-MIN
Pairing Heap Iacono 2000	$\Theta(1)$	$O(\log n)$	$\Theta(\log n)$
Pairing Heap Pettie 2005; Sinnamon 2022	$2^{O(\sqrt{\log \log N})}$	$2^{O(\sqrt{\log \log N})}$	$\Theta(\log n)$
Pairing Heap This Work	$\Theta(1)$	$(\log \log n)^{\Theta(1)^\dagger}$	$\Theta(\log n)$
Lazy Pairing Heap This Work	$\Theta(1)$	$\tilde{\Theta}(\log \log n)^\dagger$	$\Theta(\log n)$

[†]The $\Omega(\log \log n)$ lower bound on amortized decrease-key time requires insertion and delete-min to be amortized $O(\log n)$ [3].

Node Groups

Group nodes such that intra-group pairing process resembles a lazy multipass pairing process. Each lazy delete-min creates two groups:

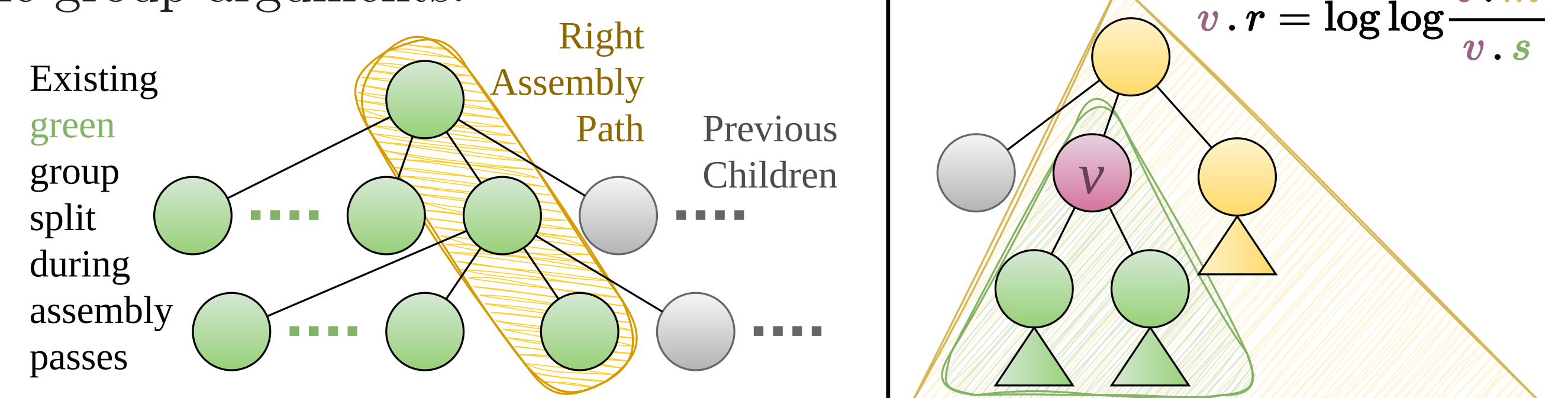


Theorem (Informal). In lazy pairing heaps, the number of inter-group pairing links is roughly $O(\log n)$ per delete-min.

Proof Sketch. Each pairing pass decrease the size of a group by a half, thus each group only participates in $\log n$ pairing passes. In each pairing pass, only the leftmost node and the rightmost node can lose an inter-group pairing link. Thus, the Theorem is implied by $2 \text{ links/pass} \times \log n \text{ passes/group} \times 2 \text{ groups/delete-min}$

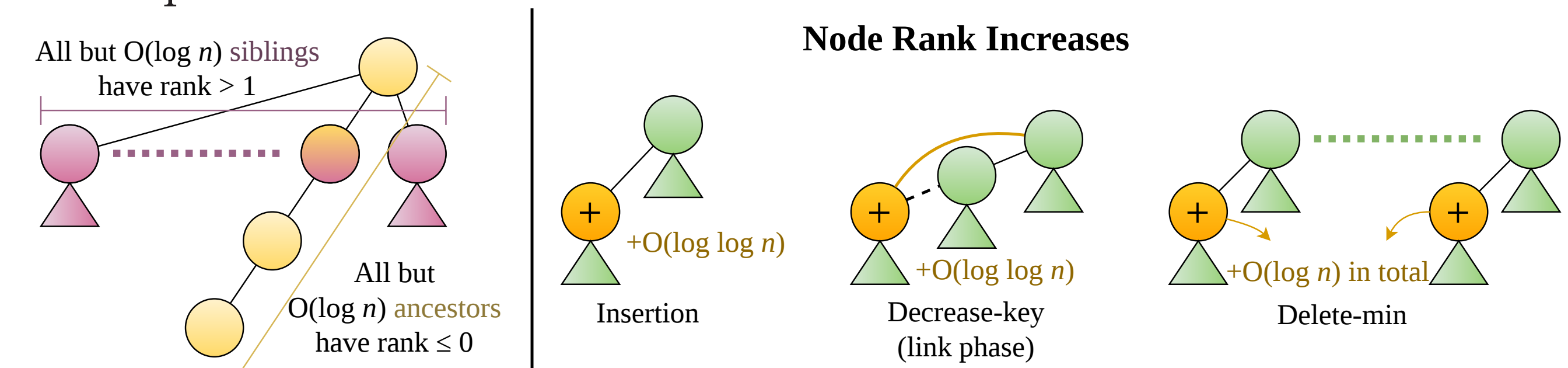
Right Assembly Path

In conventional pairing heaps, for each *right assembly link*, i.e., assembly link lost by the rightmost node, one group will be split into two. Thus we need to bound the number of right assembly links to apply the group arguments.



Theorem. The number of right assembly links is $O(1)$ per insertion plus $O(\log \log n)$ per decrease-key plus $O(\log n)$ per delete-min.

Proof Sketch. Define the rank of node v as $\log \log (v.m / v.s)$, where $v.s$ is the size of the subtree rooted at v , and $v.m$ is the total size of its parent upon the link. This definition ensures that:



Thus all but $O(\log n)$ nodes on the right assembly path decreased their rank by at least 1, which means the number of right assembly links is at most $O(\log n)$ per delete-min plus the total decreases in node ranks. The total decreases in node ranks matches the total increases, which only has the cut phase of decrease-key unbounded. We redefine the sizes and masses to bound

